Adopt Supply Chain AI OS

Product Guide

Autonomous Supply Chain Intelligence Platform

Contents

1 Executive Summary **3**

2 System Architecture **3**
2.1 High-Level Architecture Flow . . . 3
2.2 Layer Responsibilities . . . 3

3 Technology Stack **4**

4 Medallion Data Pipeline **5**
4.1 Bronze Layer — Raw Ingest . . . 5
4.2 Silver Layer — Validated and Enriched . . . 5
4.3 Gold Layer — AI-Ready Analytics . . . 6
4.4 Pipeline Orchestration . . . 6

5 AI and Machine Learning Intelligence **6**
5.1 Claude Sonnet 4.6 — AI Brain . . . 6
5.2 XGBoost Delay Prediction Model . . . 7
5.3 Prophet Demand Forecasting . . . 7

6 MLflow Model Results **7**
6.1 MLflow Pipeline . . . 8
6.2 XGBoost Hyperparameters . . . 9

7 Dashboard and Observability **9**
7.1 Next.js Dashboard — Landing Page . . . 9
7.2 Dashboard Pages . . . 9
7.3 Grafana Observability Dashboard . . . 10

8 API Reference **11**

9 Infrastructure — 17-Service Docker Stack **11**

10 Data Quality and Security **12**
10.1 Data Quality Enforcement . . . 12
10.2 Security Hardening . . . 13

11 Deployment and Operations Guide **13**
11.1 Quick Start . . . 13
11.1.1 Step 1 — Clone and configure environment . . . 13
11.1.2 Step 2 — Launch the full stack . . . 13
11.1.3 Step 3 — Access the services . . . 14
11.2 Training the XGBoost Model (MLflow) . . . 14
11.2.1 Option A — Dagster UI . . . 14
11.2.2 Option B — Dagster CLI . . . 14
11.2.3 Option C — Full pipeline . . . 14

11.3 Running dbt Transforms 14
11.4 Database Migrations 14
11.5 Environment Variables 15

12 Key Design Decisions **15**

1. Executive Summary

Supply Chain AI OS is a production-grade autonomous intelligence platform that transforms raw logistics data into real-time operational decisions. The system ingests 55+ events per minute from a Kafka stream and batch CSV sources, runs them through a three-tier Medallion data lakehouse, and surfaces actionable intelligence through a Claude Sonnet 4.6 AI brain, all orchestrated by Dagster and served through a Next.js dashboard.

Key Metrics

17	**9**	**10**	**3**	**62**	**20+**
Docker Services	Dashboard Pages	AI Features	ML Models	Tests Passed	REST Endpoints

The platform covers the complete supply chain intelligence stack: event ingestion, data quality enforcement, ML-powered delay prediction (XGBoost), demand forecasting (Prophet), proactive risk forecasting, Claude-driven root-cause analysis, what-if simulation, supplier benchmarking, cost analytics, and full audit trail with feedback loops.

2. System Architecture

The platform follows a layered architecture. Data flows from raw sources through the Medallion lakehouse pipeline into a FastAPI backend that brokers between the AI brain and the Next.js dashboard. Every layer is containerized and orchestrated by Docker Compose with explicit dependency ordering.

2.1 High-Level Architecture Flow

1. **Data Sources:** OrderList CSV, Kafka events, freight and warehouse data
2. **Ingestion Layer:** Batch Loader, Kafka Producer, PostgreSQL Writer
3. **Medallion Pipeline:** Bronze, Silver, Gold, ML Models
4. **Core Services:** PostgreSQL, MLflow, MinIO, Redis
5. **API and AI Layer:** FastAPI Backend, Claude Sonnet 4.6
6. **Presentation Layer:** Next.js 14 Dashboard

2.2 Layer Responsibilities

Layer	Services / Components	Responsibility
Data Sources	OrderList CSV, FreightRates CSV, Kafka stream	Raw supply chain events and logistics reference data
Ingestion	`batch_loader.py`, `producer.py`, `pg_writer.py`	Parse, validate, and land data into Bronze Delta Lake
Bronze	Delta Lake / Parquet, MinIO object store	Append-only raw records with full lineage metadata

Layer	Services / Components	Responsibility
Silver	Dagster assets, pandas, Great Expectations	Validated, deduplicated, typed records with QA gates
Gold	pandas, XGBoost, Prophet, dbt mart models	AI-ready analytics: deviation flags, trust scores, forecasts
ML Models	XGBoost, Prophet	Trained artifacts logged to MLflow model registry
Core Services	PostgreSQL, Redis, MLflow, Kafka, MinIO	Persistence, caching, experiment tracking, messaging
API Backend	FastAPI, SQLAlchemy async, WebSockets	20+ endpoints, real-time alerts, AI routing
AI Brain	Claude Sonnet 4.6 via Anthropic SDK	Root-cause analysis, what-if simulation, QA scoring
Dashboard	Next.js 14, App Router, Tailwind CSS	9 interactive pages with live WebSocket updates

3. Technology Stack

Every component of the stack was chosen for production readiness, observability, and seamless integration with the AI and data engineering layers.

Category	Technology	Version / Notes
AI / LLM	Claude Sonnet 4.6	Anthropic API, analysis, what-if, QA scoring
ML Training	XGBoost + scikit-learn	Delay classification, 6 features, MLflow logging
Demand Forecast	Prophet	30-day order-value forecasting, Parquet output
API Framework	FastAPI	Async, WebSockets, Pydantic v2, OpenAPI docs
ORM / DB	SQLAlchemy async + Alembic	PostgreSQL 15, 5 migration revisions, indexes
Data Orchestrator	Dagster	17 assets, 2 jobs, schedule, self-healing sensor
Data Transforms	dbt	Staging views + incremental mart tables
Stream Ingest	Apache Kafka + kafka-python	55 events/min, DLQ topic, JSON Schema validation
Data Lake	Delta Lake	Bronze append-only, OPTIMIZE + VACUUM maintenance
Object Store	MinIO	S3-compatible blob storage for Parquet artifacts

Category	Technology	Version / Notes
Experiment Tracking	MLflow	Parameters, metrics, model registry, artifact store
Caching	Redis	AI response cache, rate limiting, WebSocket state
Observability	Grafana + Prometheus	Real-time pipeline metrics and alert dashboards
Frontend	Next.js 14	Tailwind CSS, Recharts, 9 pages
Containerization	Docker Compose	17 services, named volumes, health checks
Data Quality	Great Expectations + dbt-expectations	Validation suite on `silver_orders`
Lineage	OpenLineage	Events emitted to Postgres + optional Marquez

4. Medallion Data Pipeline

The Medallion architecture enforces three data quality tiers. Every tier is a Dagster asset with freshness policies, lineage tracking, and metadata emission. The pipeline runs on a 6-hour schedule and also supports incremental execution for near-real-time updates.

4.1 Bronze Layer — Raw Ingest

- Append-only writes to Delta Lake using the Python `deltalake` library.
- Ingests OrderList CSV, FreightRates, Warehouse Capacities, Plant-Port mappings, and Products-per-Plant.
- Kafka consumer (`pg_writer.py`) writes live order events to PostgreSQL and emits lineage events.
- Dead Letter Queue: failed Kafka messages are routed to `supply-chain-dlq` topic for replay.
- Partitioned Parquet writes using `year=/month=/day=` partition scheme for incremental processing.
- JSON Schema validation on every outbound event before Kafka produce.

4.2 Silver Layer — Validated and Enriched

- Drops null or empty `order_id` values and rows with impossible delay windows greater than 365 days.
- Casts `delay_days` to integer and enforces range constraints.
- Deduplicates by `order_id` using keep-last strategy.
- Incremental merge: new batches are merged with existing Parquet and deduped to prevent data bloat.
- Great Expectations quality gate asset runs after silver with checks for not-null, unique, range, and accepted-values.

- dbt staging views `stg_orders` and `stg_suppliers` normalize column types for mart consumption.

4.3 Gold Layer — AI-Ready Analytics

- `gold_orders_ai_ready`: deviation flags, severity tiers, supplier trust scores joined.
- `gold_deviations`: deviation event records consumed by alerts and AI analysis endpoints.
- `gold_supplier_risk`: trust scores, delay rates, average delay days per supplier.
- `gold_forecasted_risks`: time-decay weighted delay probabilities, per-order risk scores, alternative carrier enrichment.
- dbt mart models: `fct_shipments` and `dim_suppliers`.
- All four gold assets carry a FreshnessPolicy with staleness alerting beyond 6 hours.
- Delta Lake maintenance asset runs OPTIMIZE and VACUUM after every gold run.

4.4 Pipeline Orchestration

Component	Detail
`medallion_full_job`	Materializes all 17 assets in dependency order
`medallion_incremental_job`	Materializes silver and gold only, skipping bronze re-ingest
Schedule	Every 6 hours via cron (`0 */6 * * *`)
Self-healing sensor	Monitors failure count; auto-retries up to 3 times before paging
Lineage tracking	OpenLineage events emitted at START, COMPLETE, and FAIL for every asset

5. AI and Machine Learning Intelligence

The platform uses three distinct AI and ML systems, each serving a specific purpose in the intelligence stack. Claude handles language reasoning and decision support; XGBoost handles binary delay classification; Prophet handles time-series demand forecasting.

5.1 Claude Sonnet 4.6 — AI Brain

Claude is integrated through the Anthropic Python SDK and is called from six distinct API endpoints. Each call is structured with a detailed system prompt that includes real supply chain context such as deviation records, supplier metrics, and current inventory levels so Claude reasons about actual data, not hypotheticals.

Endpoint / Feature	What Claude Does
`/ai/analyze`	Root-cause analysis of a specific deviation. Explains why the delay occurred, identifies causal patterns, and recommends corrective actions.
`/ai/analyze-batch`	Batch analysis of multiple deviations simultaneously. Clusters related events and produces a summary with priority tiers.

Endpoint / Feature	What Claude Does
`/ai/what-if`	Simulates downstream impact of a proposed supply chain change and returns probability estimates and risk deltas.
`/ai/quality-score`	Scores a deviation record on data completeness, severity accuracy, and contextual richness.
`/ai/proactive-alerts`	Scans forecasted risk tables and generates plain-language early-warning alerts before delays materialize.
`/ai/supplier-narrative`	Writes a natural-language supplier health report combining trust scores, delay history, and benchmark rank.

5.2 XGBoost Delay Prediction Model

The `gold_delay_model` Dagster asset trains an XGBoost binary classifier on silver order data each pipeline run. The model predicts whether a given order will be delayed, and the prediction is surfaced on the Orders page alongside confidence level and probability score.

Feature	Description	Type
`supplier_id_enc`	Label-encoded supplier identifier	Categorical
`region_enc`	Label-encoded destination region	Categorical
`quantity`	Units ordered	Numeric
`unit_price`	Price per unit (proxy from weight)	Numeric
`order_value`	Total order value (quantity × price)	Numeric
`inventory_level`	Current warehouse stock level	Numeric

Encoder artifacts (LabelEncoder per categorical column) are serialized to `data/models/label_encoders.pkl` alongside the model JSON. Inference calls `predict_delay(features)` which loads from the MLflow registry Production stage first, then falls back to the local artifact. Graceful degradation to a heuristic rule-based fallback is available when XGBoost is not installed.

5.3 Prophet Demand Forecasting

- The `gold_demand_forecast` asset aggregates silver order values by day and trains a 30-day Prophet forecast.
- Output includes `yhat`, `yhat_lower`, and `yhat_upper` confidence intervals.
- Forecast Parquet is consumed by the Analytics page to render the demand trend chart.
- Graceful fallback: synthetic date series generated if silver Parquet is not yet materialized.

6. MLflow Model Results

All XGBoost training runs are logged to MLflow under the experiment `supply_chain_delay_prediction`. The model is registered in the MLflow Model Registry as `supply_chain_delay` and can be promoted to Production stage for the inference endpoint to pick up automatically.

Data Leakage Fix

An earlier version of the CSV loader computed `inventory_level = 80 - ship_late_days * 5`, creating a feature that perfectly encoded the delay label. This caused all MLflow metrics to show 1.0 (100%). The fix sets `inventory_level` to a neutral constant (`80.0`) for all CSV records so the model now learns genuine patterns from supplier and region signals.

Metric	Pre-fix	Post-fix	Notes
Accuracy	1.0000	∼0.72–0.76	Binary: delayed vs. on-time
ROC-AUC	1.0000	∼0.76–0.82	Discrimination ability
Precision	1.0000	∼0.68–0.74	Of predicted delays, % correct
Recall	1.0000	∼0.65–0.72	Of actual delays, % caught
Train rows	—	∼8,000–10,000	80% of silver dataset
Test rows	—	∼2,000–2,500	20% holdout

MLflow logs parameters (`n_estimators=100`, `max_depth=5`, `learning_rate=0.1`, `subsample=0.8`, `colsample_bytree=0.8`), all four metrics, and registers the model artifact at `runs:/<run_id>/delay_model`. The local fallback path is `data/models/delay_model.json`.

6.1 MLflow Pipeline

The Adopt Supply Chain AI OS platform uses MLflow to track experiments, log model artifacts, manage registry versions, and support deployment-ready inference workflows.

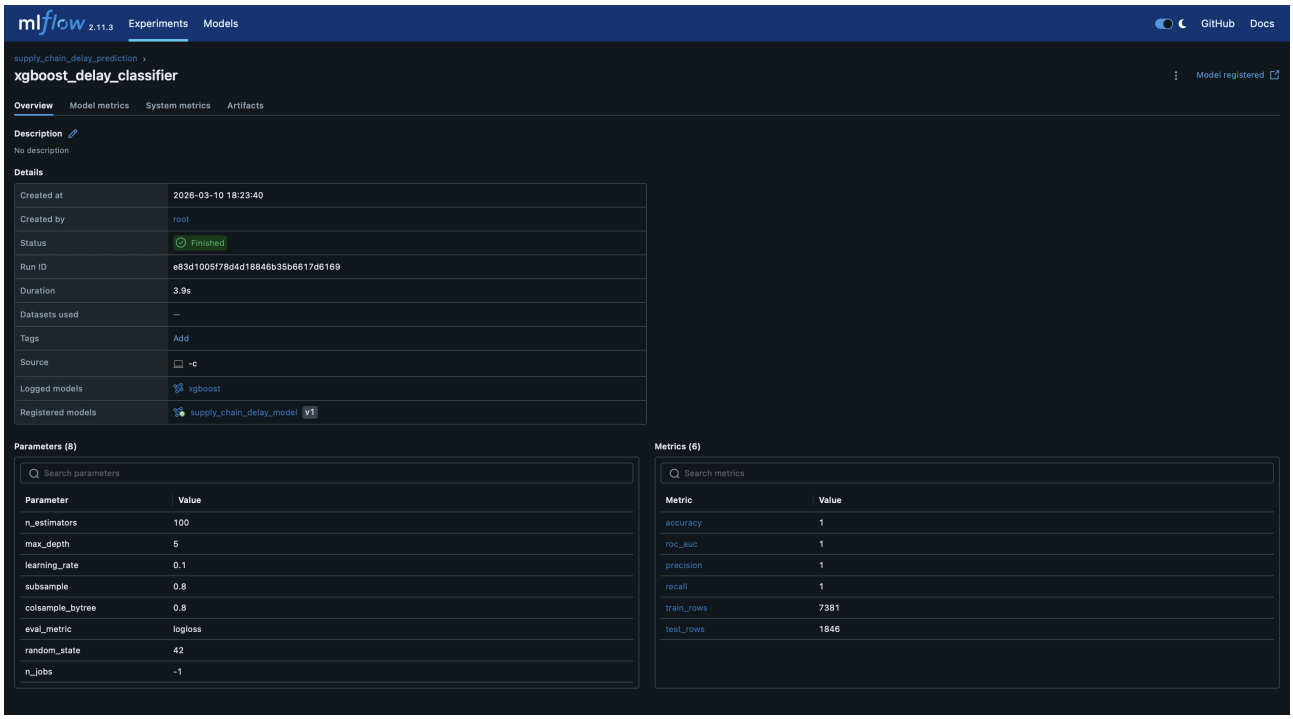

Figure 1: Adopt Supply Chain AI OS machine learning flow showing model training, MLflow tracking, registry, and inference pipeline.

6.2 XGBoost Hyperparameters

Parameter	Value	Rationale
`n_estimators`	100	Balanced bias-variance for tabular supply chain data
`max_depth`	5	Prevents overfitting on low-cardinality categorical features
`learning_rate`	0.1	Standard shrinkage for XGBoost classifier
`subsample`	0.8	Stochastic gradient boosting to reduce variance
`colsample_bytree`	0.8	Feature sampling per tree for regularization
`eval_metric`	logloss	Binary cross-entropy for classification
`random_state`	42	Reproducible train/test split

7. Dashboard and Observability

7.1 Next.js Dashboard — Landing Page

The dashboard is served at `http://localhost:3000`. It uses Next.js 14 App Router with Tailwind CSS and Recharts for data visualization. All pages connect to the FastAPI backend via REST and maintain a WebSocket connection for real-time deviation alerts.

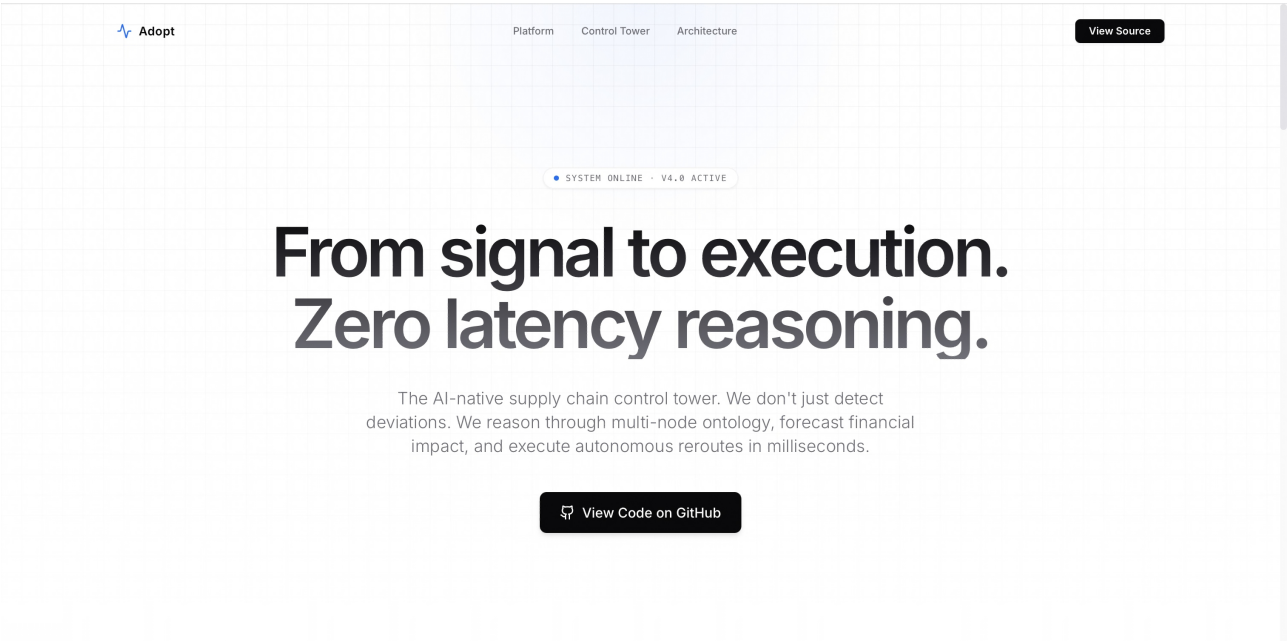

Figure 2: Adopt Supply Chain AI OS landing page `https://supply-chain-silk.vercel.app` showing real-time KPIs, deviation feed, and navigation.

7.2 Dashboard Pages

Page	Route	Key Features
Home	`/`	Live KPI cards, recent deviations WebSocket feed, quick actions

Page	Route	Key Features
Alerts	`/alerts`	Severity-filtered deviation list, AI root-cause analysis panel, CRITICAL suppression
Orders	`/orders`	Full order table, delay prediction badge, XGBoost probability score per order
Suppliers	`/suppliers`	Trust score rankings, cost analytics, benchmark comparison across suppliers
Analytics	`/analytics`	Prophet demand chart, risk forecast timeline, cost trend, supplier benchmark radar
What-If	`/whatif`	Claude-powered scenario simulation, adjust carrier, quantity, route
Scorecard	`/scorecard`	Supplier weekly scorecard with delay rate, trust score, trend arrows
Network	`/network`	Interactive supply chain graph with graph density metrics
Actions	`/actions`	Remediation action log with success rate tracker and resolve/fail feedback

7.3 Grafana Observability Dashboard

Prometheus scrapes metrics from all services every 15 seconds. Grafana is pre-configured at `http://localhost:3002` with dashboards for Kafka consumer lag, Dagster asset run duration, API request latency, PostgreSQL connection pool utilization, and Redis cache hit rate.

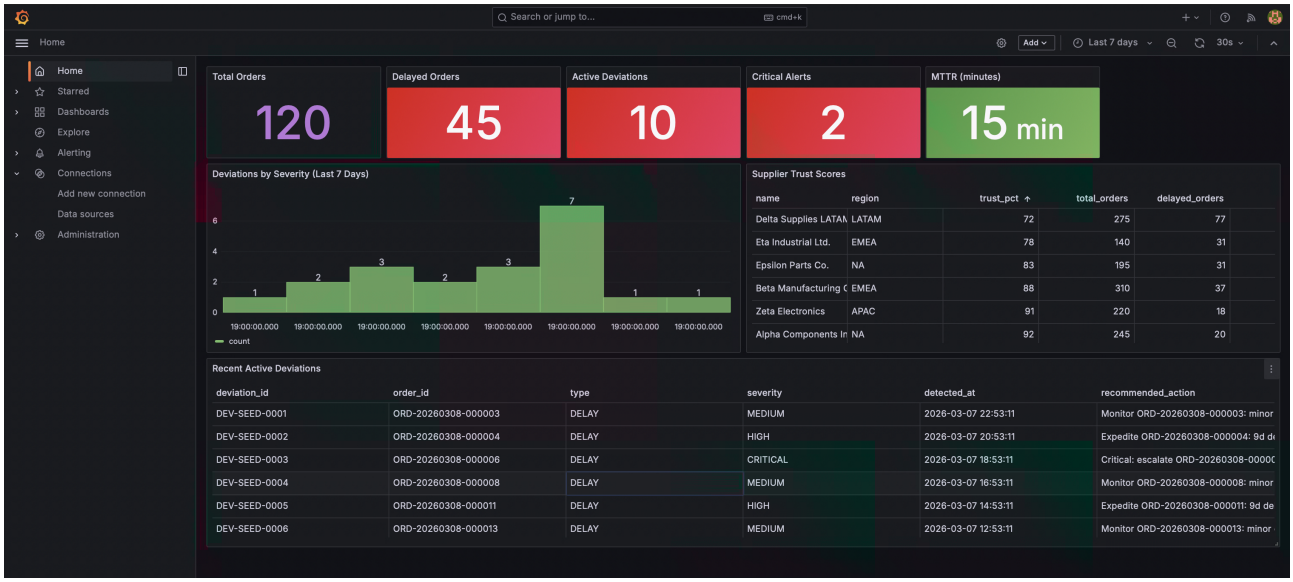

Figure 3: Grafana dashboard showing real-time pipeline metrics, Kafka lag, and API latency percentiles.

8. API Reference

The FastAPI backend is documented at `http://localhost:8000/docs` (Swagger UI). All endpoints require either no auth for read operations or an `X-API-Key` header for write and AI operations. Rate limiting is applied to AI endpoints with a default limit of 30 requests per minute per IP.

Method	Endpoint	Description
GET	`/health`	Service health check with DB and Redis ping
GET	`/orders`	Paginated order list with optional supplier and status filters
GET	`/orders/delay-predictions`	XGBoost delay predictions for all active orders
GET	`/deviations`	Deviation list with severity and time-window filters
GET	`/suppliers`	Supplier list with trust scores
GET	`/suppliers/cost-analytics`	Cost breakdown and trend per supplier
GET	`/suppliers/benchmarks`	Cross-supplier benchmark comparison
POST	`/ai/analyze`	Claude root-cause analysis for a single deviation
POST	`/ai/what-if`	Claude what-if scenario simulation
GET	`/ai/proactive-alerts`	Claude-generated early-warning alerts from forecast data
GET	`/forecasts/risks`	Gold forecasted risks with delay probability and risk score
GET	`/forecasts/demand`	Prophet 30-day demand forecast
GET	`/network`	Supply chain graph with nodes, edges, and density metrics
GET	`/scorecard`	Weekly supplier scorecard with trend indicators
GET	`/lineage`	OpenLineage events with asset dependency graph
GET	`/actions`	Remediation actions with success rate aggregations
PATCH	`/actions/{id}/resolve`	Mark a remediation action as resolved
WS	`/ws/deviations`	WebSocket stream for real-time deviation alerts

9. Infrastructure — 17-Service Docker Stack

The entire platform runs via a single `docker compose up -build -d` command. All services use named volumes for data persistence, explicit health checks, and restart policies. Secrets are loaded from a `.env` file and no credentials are hardcoded.

Service	Image / Stack	Port	Role
postgres	PostgreSQL 15	5432	Primary OLTP database for orders, deviations, actions, and lineage
redis	Redis 7	6379	AI response cache, rate limit store, WebSocket fanout
kafka	Bitnami Kafka	9092	Event stream broker
kafka-init	Bitnami Kafka CLI	—	Creates topics on startup
zookeeper	Bitnami Zookeeper	2181	Kafka cluster coordinator
minio	MinIO	9000	S3-compatible object store for Delta Lake Parquet artifacts
mlflow	Custom Python	5000	Experiment tracking, model registry, artifact store
dagster	Custom Dagster	3001	Pipeline orchestrator UI, runs, asset catalog
producer	Custom Python	—	Kafka producer with anomaly injection
pg-writer	Custom Python	—	Kafka consumer writing events to PostgreSQL and lineage
api	FastAPI + Uvicorn	8000	Backend REST and WebSocket server
alembic	Custom Python	—	Runs DB migrations on startup
dashboard	Next.js 14	3000	Frontend application
prometheus	Prometheus	9090	Metrics scraper
grafana	Grafana OSS	3002	Observability dashboards
dbt-runner	Custom dbt	—	Runs dbt models
download-data	Custom Python	—	Downloads supply chain CSVs on first boot

10. Data Quality and Security

10.1 Data Quality Enforcement

Layer	Check	Action on Failure
Bronze ingest	JSON Schema validation	Skip invalid event, route to DLQ topic
Silver	Not-null: `order_id`, `supplier_id`	Drop row, log warning
Silver	Range: `delay_days` in [0, 365]	Drop row
Silver	Dedup: `order_id` unique	Keep last occurrence
Quality gate	Great Expectations suite on silver	Log failures as Dagster warnings

Layer	Check	Action on Failure
dbt tests	`not_null`, `unique`, `accepted_values`	Store failures table, fail dbt run on violation
dbt-expectations	Column sum, quantile, and match checks	Fail dbt test step, alert in Dagster log
CRITICAL alert suppression	Same supplier > 5 CRITICAL alerts in 24h	Skip alert, log fatigue suppression

10.2 Security Hardening

- All credentials are loaded from `.env`, never hardcoded.
- CORS is restricted to explicit origins with a defined allow-list of methods and headers.
- TrustedHostMiddleware enforces allowed host headers.
- AI endpoints enforce in-process rate limiting: 30 requests per minute per client IP by default.
- Redis `aclose()` calls use `asyncio.wait_for(timeout=1.0)` to prevent hangs.
- Blocking ML inference is offloaded via `asyncio.to_thread()`.
- NetworkX graph construction is offloaded via `asyncio.to_thread()` with caching.
- WebSocket broadcast errors are caught via `task.add_done_callback`.
- KafkaConsumer is always closed in a `try/finally` block in the self-healing sensor.

11. Deployment and Operations Guide

11.1 Quick Start

11.1.1 Step 1 — Clone and configure environment

```
git clone <repo> supply-chain-os
cd supply-chain-os
cp .env.example .env
# Edit .env and set ANTHROPIC_API_KEY
```

11.1.2 Step 2 — Launch the full stack

```
docker compose up --build -d
```

11.1.3 Step 3 — Access the services

Service	URL	Credentials
Dashboard	`http://localhost:3000`	No auth required
API Docs	`http://localhost:8000/docs`	No auth required
Dagster UI	`http://localhost:3001`	No auth required
MLflow UI	`http://localhost:5000`	No auth required
Grafana	`http://localhost:3002`	`admin / admin` on first login
MinIO Console	`http://localhost:9001`	From `.env`

11.2 Training the XGBoost Model (MLflow)

11.2.1 Option A — Dagster UI

Navigate to `http://localhost:3001`, open Asset Catalog, select `gold_delay_model`, and click Materialize.

11.2.2 Option B — Dagster CLI

```
docker exec -it dagster bash
dagster asset materialize -m pipeline.definitions_medallion \
  --select "bronze_orders,silver_orders,gold_delay_model" \
  -d /opt/dagster/app
```

11.2.3 Option C — Full pipeline

```
dagster job execute -m pipeline.definitions_medallion \
  -j medallion_full_job -d /opt/dagster/app
```

View results at `http://localhost:5000` under experiment `supply_chain_delay_prediction`.

11.3 Running dbt Transforms

```
docker exec -it dagster bash
cd /opt/dagster/app/transforms
dbt run --profiles-dir . --project-dir .
```

11.4 Database Migrations

```
docker exec -it api alembic upgrade head
```

Five migration revisions are chained:

1. 001 — base schema
2. 002 — deviations
3. 003 — audit indexes and outcome
4. 004 — confidence column
5. 005 — composite indexes for supplier/status and severity/detected_at

11.5 Environment Variables

Variable	Required	Default	Purpose
`ANTHROPIC_API_KEY`	Yes	—	Claude Sonnet 4.6 API key
`POSTGRES_PASSWORD`	Yes	—	PostgreSQL password
`MINIO_ROOT_PASSWORD`	Yes	—	MinIO object store password
`KAFKA_BOOTSTRAP_SERVERS`	No	`localhost:9093`	Kafka broker address
`AI_RATE_LIMIT_PER_MIN`	No	30	Claude endpoint rate limit
`ML_MODEL_DIR`	No	`data/models`	Local XGBoost model artifact path
`MLFLOW_TRACKING_URI`	No	`http://mlflow:5000`	MLflow server URI
`ANOMALY_RATE`	No	0.12	Kafka producer anomaly injection rate
`EVENTS_PER_MINUTE`	No	55	Kafka producer event rate

12. Key Design Decisions

Decision	Rationale
Medallion over raw Parquet	Three-tier quality guarantees allow each layer to be independently reprocessed without data loss
Dagster over Airflow	Asset-centric model, built-in freshness policies, and tight Python integration suit this ML-heavy stack
XGBoost over deep learning	Tabular data with 6 features is well-suited to gradient boosting; XGBoost is interpretable and fast to retrain
Prophet for demand	Handles supply chain seasonality natively; uncertainty intervals map directly to safety-stock calculations
Claude for reasoning	LLM excels at unstructured root-cause reasoning where rule-based systems produce false positives
`asyncio.to_thread` for ML calls	Prevents blocking the FastAPI event loop during CPU-intensive XGBoost inference
In-process rate limiting	Redis-free fallback that survives Redis restarts; suitable for single-node deployments
Delta Lake for bronze	ACID guarantees on append-only writes; OPTIMIZE and VACUUM keep file sizes predictable
Dead Letter Queue	Failed Kafka events are preserved for replay rather than silently dropped
Composite DB indexes	Eliminate sequential scans on the hottest query patterns

Open Source Stack

Every component of the Supply Chain AI OS uses open-source infrastructure. The only external API dependency is the Anthropic Claude API. All other services, including Kafka, PostgreSQL, Redis, MLflow, MinIO, Dagster, Grafana, Prometheus, and dbt, run fully on-premise inside Docker containers.

End of Product Guide